



**INTELLIGENT FRAUD DETECTION
MODEL FOR SECURING
ONLINE PAYMENT TRANSACTION**



A DESIGN PROJECT REPORT

Submitted by

AJAY KUMAR.S

DINESH.M

VIGNESH.S

in partial fulfillment for the award of the degree

of

BACHELOR OF TECHNOLOGY

in

ARTIFICIAL INTELLIGENCE AND DATA SCIENCE

K. RAMAKRISHNAN COLLEGE OF TECHNOLOGY

(An Autonomous Institution, affiliated to Anna University Chennai and Approved by AICTE,
New Delhi)

SAMAYAPURAM, TIRUCHIRAPPALLI-621112

DECEMBER 2024

K. RAMAKRISHNAN COLLEGE OF TECHNOLOGY
(AN AUTONOMOUS INSTITUTION)

Affiliated to Anna University Chennai and Approved by AICTE, New Delhi

BONAFIDE CERTIFICATE

Certified that this design project report titled **“INTELLIGENT FRAUD DETECTION MODEL FOR SECURING ONLINE PAYMENT TRANSACTION”** is the Bonafide work of **AJAYKUMAR.S (REG NO: 811722243002), DINESH.M (REG NO: 811722243014), VIGNESH.S (REG NO: 811722243053)** who carried out the project work under my supervision.

SIGNATURE

DR. T. AVUDAIAPPAN, M.E., PH.D.,
HEAD OF THE DEPARTMENT

Associate Professor,
Department of Artificial Intelligence,
K. Ramakrishnan College of Technology
(Autonomous),
Samayapuram – 621 112.

SIGNATURE

Mr.T.SHYAMSUNDAR M.E.,
SUPERVISOR

Assistant Professor,
Department of Artificial Intelligence,
K. Ramakrishnan College of Technology
(Autonomous),
Samayapuram – 621 112.

Submitted for the viva-voce examination held on

INTERNAL EXAMINER

EXTERNAL EXAMINER

DECLARATION

We jointly declare that the project report on “**INTELLIGENT FRAUD DETECTIONMODEL FOR SECURING ONLINE PAYMENT TRANSACTION**” is the result of original work done by us and best of our knowledge, similar work has not been submitted to “**ANNA UNIVERSITY CHENNAI**” for the requirement of Degree of **BACHELOR OF TECHNOLOGY**. This project report is submitted on the partial fulfilment of the requirement of the award of Degree of **BACHELOR OF TECHNOLOGY**.

SIGNATURE

AJAY KUMAR.S

DINESH.M

VIGNESH.S

PLACE: SAMAYAPURAM

DATE:

ACKNOWLEDGEMENT

It is with great pride that we express our gratitude and in - debt to our institution “**K. Ramakrishnan College of Technology (KRCT) (Autonomous)**”, for providing us with the opportunity to do this project.

We are glad to credit honorable Chairman **Dr. K. RAMAKRISHNAN, B.E.**, for having provided for the facilities during the course of our study in college.

We would like to express our sincere thanks to our beloved Executive Director **Dr. S. KUPPUSAMY, MBA, Ph.D.**, for forwarding to our project and offering adequate duration in completing our project.

We would like to thank our Principal **Dr. N. VASUDEVAN, M.E., Ph.D.**, who gave opportunity to frame the project the full satisfaction.

We whole heartily thanks to **Dr. T. AVUDAIAPPAN, M.E., Ph.D.**, Head of the Department, **ARTIFICIAL INTELLIGENCE** for providing his encourage pursuing this project.

We express our deep and sincere gratitude to our Project Guide, **Mr. T.SHYAMSUNDAR, M.E.**, Assistant Professor of **ARTIFICIAL INTELLIGENCE**, for her incalculable suggestions, creativity, assistance, and patience which motivated us to carry it.

We render our sincere thanks to our Project Coordinator **Mrs. M. A. REETHA JEYARANI, M.E.**, and other staff members for providing valuable information during the course.

We wish to express our special thanks to the officials and Lab Technicians of our departments who rendered their help during the period of the work progress.

ABSTRACT

The "Online Transaction Fraud Detection" system is a machine learning-based solution designed to combat financial fraud by leveraging historical transaction data and advanced algorithms like logistic regression, random forests, and neural networks. This adaptive model learns to differentiate between legitimate and fraudulent transactions, offering a highly accurate, real-time detection system that evolves with emerging fraud tactics. By quickly identifying suspicious activity, the system helps financial institutions and customers prevent financial losses while minimizing false alarm disruptions. The solution aims to improve security and efficiency in an increasingly digital financial landscape. Additionally, it continuously adapts to new fraud patterns, ensuring long-term effectiveness in a rapidly evolving environment. Additionally, the system integrates seamlessly with existing payment platforms, providing real-time alerts and immediate actions to prevent fraud. This proactive approach enhances financial security, helping both financial institutions and customers avoid significant losses.

TABLE OF CONTENTS

CHAPTER NO.	TITLE	PAGE NO.
	ABSTRACT	v
	LIST OF FIGURES	ix
	LIST OF ABBEVIATIONS	x
1	INTRODUCTION	1
	1.1 OBJECTIVE	1
2	LITERATURE SURVEY	2
	2.1 FRAUD DETECTION USING MACHINE LEARNING ALGORITHMS	2
	2.2 MACHINE LEARNING FOR ONLINE TRANSACTION FRAUD DETECTION SYSTEM	3
	2.3 CREDIT CARD FRAUD DETECTION USING RANDOM FOREST ALGORITHM	4
	2.4 CREDIT DATA FRAUD DETECTION USING KERNEL METHODS WITH SVM	5
3	SYSTEM ANALYSIS	6
	3.1 EXISTING SYSTEM	6
	3.1.1. SYSTEM	6
	3.1.2. BASE DOMAIN	6
	3.1.3. TOOL USED	6
	3.1.4. MERITS	6

3.1.5.	DEMERITS	6
3.2	PROPOSED SYSTEM	7
3.2.1	SYSTEM	7
3.2.2	BASE DOMAIN	7
3.2.3	TOOL USED	7
3.2.4	ADVANCEMENTS	7
3.2.5	MAIN GOALS	7
4	SYSTEM SPECIFICATION	9
4.1	HARDWARE SPECIFICATION	9
4.1.1	HARDWARE DESCRIPTION	9
4.2	SOFTWARE SPECIFICATION	10
4.2.1	SOFTWARE DESCRIPTION	10
5	SYSTEM DESIGN	11
5.1	DATA FLOW DIAGRAM	12
5.2	SYSTEM ARCHITECTURE	13
5.3	USE-CASE DIAGRAM	14
5.4	CLASS DIAGRAM	15
6	MODULE DESCRIPTION	16
6.1	DATA COLLECTION	16
6.2	DATA PREPROCESSING	16
6.3	SYNTHETIC DATA GENERATION	17
6.4	MODEL (RANDOM FOREST)	17

6.5	MODEL TRAINING	18
6.6	MODEL EVALUATION	19
6.7	MODEL PREDICTION	19
6.8	VISUALIZATION	20
7	CONCLUSION AND FUTURE ENHANCEMENT	21
7.1	CONCLUSION	21
7.2	FUTURE ENHANCEMENT	21
	APPENDIX 1 SAMPLE CODE	22
	APPENDIX 2 SCREENSHOTS	39
	REFERENCES	44

LIST OF FIGURES

FIGURE NO	TITLE	PAGE NO
5.1	DATA FLOW DIAGRAM	12
5.2	SYSTEM ARCHITECTURE	13
5.3	USE-CASE DIAGRAM	14
5.4	CLASS DIAGRAM	15
A.2.1	DATASETS	39
A.2.2	MODEL EVALUATION AND HYPER PARAMETER TUNING	40
A.2.3	VISUALIZATION	40
A.2.4	HOME PAGE	41
A.2.5	PDF UPLOADED ACKNOWLEDGEMENT PAGE	41

LIST OF ABBREVIATIONS

SVM	Support Vector machine
TAN	Tree Augmented Naive Bayes
SMO	Sequential Minimal Optimization
C-SVC	C-Support Vector Classification
LIBSVM	Library for Support Vector Machine
GUI	Graphical User Interface
XAI	Explainable Artificial Intelligence
RNN	Recurrent Neural Network
SMOTE	Synthetic Minority Over Sampling Technique

CHAPTER 1

INTRODUCTION

In today's digital world, credit card fraud has become a serious and growing threat, impacting both individuals and businesses. Traditional fraud detection methods, which rely on fixed rules and manual reviews, struggle to keep up with the speed at which fraud tactics evolve. As a result, they often miss new types of fraud or mistakenly flag legitimate transactions, frustrating customers. Machine learning offers a solution by allowing fraud detection systems to "learn" from historical data, adapting to new patterns and catching fraud in real-time. This approach enhances accuracy, minimizes false positives, and creates a smoother experience for users, helping both banks and customers stay protected.

1.1 OBJECTIVE

The objective of this credit card fraud detection system is to create a highly accurate, adaptable, and efficient way to detect fraud using the Random Forest algorithm. Random Forest is a powerful machine learning technique that combines multiple decision trees to make more accurate predictions. By analyzing past transaction data, the Random Forest algorithm learns the unique patterns that differentiate legitimate purchases from fraudulent ones. This approach improves detection accuracy and significantly reduces false positives, allowing legitimate transactions to go through smoothly.

Our goal is to leverage the Random Forest algorithm's strengths in handling complex datasets and detecting subtle fraud patterns that simpler methods might miss. By continuously training and updating the model, the system can quickly adapt to new types of fraud, helping financial institutions protect their customers more effectively and enhance trust in their services.

CHAPTER 2

LITERATURE SURVEY

2.1 TITLE: Fraud Detection using Machine Learning Algorithms

Authors: Vaishnavi Nath Dornadula and Geetha S

Year of Publication: 2012

Techniques used : Clustering method Sliding-Window method

Abstract:

Frauds are increasingly common, particularly with the rise of e-commerce and online payment systems, making online transactions vulnerable to fraud. This study aims to develop a novel fraud detection method tailored for streaming transaction data. It leverages past transaction records to identify behavioral patterns of customers by grouping them based on transaction amounts. A sliding window strategy is applied to analyze these grouped transactions, enabling the extraction of behavioral patterns for each group. Various classifiers are trained on these groups separately, allowing for the identification of the most effective classifier based on rating scores. A feedback mechanism is also integrated to address the challenge of concept drift. This research uses a European credit card fraud dataset to validate the proposed approach

Merits:

- Enhanced Fraud Detection Accuracy

Demerits:

- Imbalanced Data Issue :Struggles with fraud detection in highly imbalanced datasets.

2.2 TITLE: Machine Learning For Online Transaction Fraud Detection System

Authors: Lakshmi S V S S, Selvani Deepthi Kavila

Year of Publication: 2018

Techniques used :: Logistic Regression, Decision Trees, Random Forest

Abstract:

The rapid participation in online-based transactional activities has increased fraudulent cases worldwide, causing significant losses to individuals and the financial industry. Among various criminal activities in the financial sector, credit card fraud is particularly prevalent and concerning for online customers. Countering these fraud activities through data mining and machine learning is a prominent approach that scholars have introduced to prevent such losses. Data mining techniques are employed to study the patterns and characteristics of suspicious and non-suspicious transactions, using normalized and anomalous data. Machine learning (ML) techniques, on the other hand, predict suspicious and non-suspicious transactions automatically using classifiers. This paper discusses supervised classification using Bayesian network classifiers (K2, Tree Augmented Naïve Bayes (TAN), Naïve Bayes), logistic regression, and J48 classifiers. After preprocessing the dataset using normalization and Principal Component Analysis, all classifiers achieved over 95% accuracy compared to results obtained without preprocessing.

Merits:

- High Detection Accuracy

Demerits:

- Overfitting (Decision Trees): Prone to overfitting with noisy data

2.3 TITLE: Credit Card Fraud Detection using Random Forest Algorithm

Author: Vaishnave Jonnalagadda, Priya Gupta, Eesita Sen

Year of Publication: 2019

Techniques Used: The main technique used in this paper is the Random Forest Algorithm.

Abstract:

The rapid growth in the e-commerce industry has led to an exponential increase in credit card usage for online purchases, consequently resulting in a surge in fraud. Detecting fraud in credit card systems has become challenging for banks. Machine learning plays a crucial role in detecting fraudulent transactions. Banks utilize various machine learning methodologies, collecting past data and incorporating new features to enhance predictive accuracy. Fraud detection performance depends significantly on the data sampling approach, variable selection, and detection techniques employed. This paper evaluates the performance of logistic regression, decision tree, and random forest methods for credit card fraud detection. The dataset, containing 284,808 credit card transactions from a European bank, is highly imbalanced, with only 0.172% fraud transactions. The dataset was oversampled to achieve a 60-40 balance between fraud and genuine transactions. Results indicate that random forest outperforms logistic regression and decision tree classifiers, achieving an accuracy of 95.5%.

Merits:

- Effective Handling of Imbalanced Data

Demerits:

- Requires careful tuning of parameters for optimal Performance, May not scale well with extremely large datasets

2.4 TITLE: Credit Data Fraud Detection using Kernel Methods with svm

Authors: Maryamsadat Hejazi, Yashwant Prasad Singh

Year of Publication: 2019

Techniques Used: KNN,SMO,SVC

Abstract:

This paper presents binary Support Vector Machine classifiers for detection of fraudulent credit card transactions. Support Vector Machine (SVM) is a well-known method in statistical learning theory based on the principle of structural risk minimization for classification and regression problems. The objective of this paper is about evaluation of accuracy and performance of different kernel methods using Sequential Minimal Optimization (SMO), C-Support Vector Classification (C-SVC) and -Support Vector Classification (-SVC) for fraudulent credit card transaction detection. A Comparison of computation time and classifiers' accuracy for SMO classifier with C-SVC in LIBSVM is provided. The simulation results show that accuracy of SMO classifier with different kernel functions is higher than C-SVC on credit data dataset obtained from UCI machine learning database. Vector Classification; LIBSVM; SMO.

Merits:

- Performance Comparison

Demerits:

- SVMs with kernel methods can be slow and resource-heavy for large datasets

CHAPTER 3

SYSTEM ANALYSIS

3.1 EXISTING SYSTEM

- **System:** Traditional rule-based fraud detection.
- **Base Domain:** Basic decision-making based on pre-defined rules.
- **Tools Used:** Basic statistical models, manual reviews.
- **Merits:**
 - Simplicity in design and implementation.
 - No need for large datasets.
- **Demerits:**
 - High false-positive rates.
 - Limited adaptability to new fraud methods.
 - Slower detection speed compared to automated systems

The existing system for fraud detection relies on traditional rule-based methodologies, which utilize basic decision-making through pre-defined rules, supported by simple statistical models and manual reviews. This system is straightforward in design and implementation, requiring no extensive datasets or complex infrastructure. However, it is characterized by significant limitations, including a high rate of false positives, which frequently flags legitimate transactions as fraudulent. Additionally, the system's static nature makes it inflexible and unable to adapt to emerging or evolving fraud tactics. The dependency on manual reviews also slows down the detection and resolution process, making it less efficient for real-time applications. Furthermore, this approach struggles to scale effectively with increasing transaction volumes and lacks the sophistication to identify complex or subtle fraudulent activities.

3.2 PROPOSED SYSTEM

- **System:** Machine Learning-based online transactional Fraud Detection.
- **Base Domain:** Supervised and unsupervised learning algorithms in fraud detection.
- **Tools Used:** Python, scikit-learn, TensorFlow, data preprocessing and annotation tools.
- **Advancements:**
 - Real-time fraud detection.
 - High precision and adaptability to evolving fraud patterns.
- **Main Goals:**
 - Continuous monitoring and detection of fraudulent transactions.
 - Minimized false positives to ensure smooth customer experience.
 - Enhanced adaptability through continuous data-driven learning

The proposed system is a **machine learning-based online transactional fraud detection framework** designed to identify and prevent fraudulent activities in real time. By leveraging both supervised and unsupervised learning algorithms, the system aims to provide a robust solution to the dynamic and evolving nature of online fraud. Supervised learning techniques will utilize labeled transactional data to train models that accurately classify legitimate and fraudulent transactions, while unsupervised methods will focus on detecting anomalies in unlabeled data to identify new and previously unseen fraud patterns. This dual approach ensures comprehensive coverage of fraud detection scenarios.

The system will be implemented using modern tools such as **Python** and **scikit-learn** along with data preprocessing and annotation tools to clean, transform, and label data for efficient model training. These tools enable the use of advanced machine learning models and deep learning techniques, ensuring high precision and adaptability in detecting evolving fraud tactics. Additionally, the system will incorporate real-time monitoring capabilities, enabling it to analyze transactions as they occur and provide immediate responses to suspicious activities.

A key advancement of the proposed system is its ability to adapt to new fraud patterns without requiring extensive manual intervention. By leveraging continuous data-driven learning, the system will update itself based on new transactional data, making it resilient to changing fraud tactics.

CHAPTER 4

SYSTEM SPECIFICATION

4.1 HARDWARE SPECIFICATIONS

- i)Processor.
- ii)Storage.
- iii)Memory (RAM).

4.1.1 HARDWARE DESCRIPTIONS

The hardware setup for this project is designed to ensure efficient performance and seamless execution of data-intensive tasks such as data preprocessing, model training, and evaluation. A powerful processor, such as an Intel Core i5 or AMD Ryzen 5, is essential for handling computationally heavy operations, providing multi-core processing and high clock speeds for faster execution. Storage requirements include a 256 GB SSD for quick data access and system responsiveness, complemented by a 1 TB HDD for storing larger datasets and backups. Adequate memory is also crucial, with 8 GB RAM serving as a minimum requirement, while 16 GB or more is ideal for managing large datasets and simultaneous processing tasks without performance bottlenecks. This hardware configuration ensures that the system can handle the demands of fraud detection modeling and analysis efficiently.

4.2 SOFTWARE SPECIFICATIONS

1. Operating System:

- **i)Windows (optional):** Windows 10/11 or Windows Server for environments requiring GUI based solutions

2. Programming Languages:

- **Python:** Primary language for Random Forest implementation.

-Libraries: Scikit-learn, NumPy, pandas.

3. Machine Learning Libraries:

- **Scikit-learn (Python):** Built-in Random Forest implementation.

4. Tools for Development and Deployment:

- **IDEs:** VSCode
- **Version Control:** Git for code versioning.

5. Deployment Tools:

- **Flask :** For building APIs to deploy the model.

4.2.1 SOFTWARE DESCRIPTIONS

The software setup for this project integrates essential tools and platforms to ensure efficient development, training, and deployment of the fraud detection system. The operating system can be either Windows 10/11 or Windows Server, providing a reliable environment for GUI-based solutions. Python is the primary programming language, offering robust libraries like Scikit-learn, NumPy, and pandas, which simplify data manipulation and machine learning tasks. Scikit-learn is particularly utilized for its built-in Random Forest implementation through classes like RandomForestClassifier. Development is streamlined using Visual Studio Code

(VSCode), an advanced Integrated Development Environment (IDE), while Git is employed for version control to maintain and track code changes. For deployment, Flask is leveraged to build APIs that allow seamless integration of the trained model into user-facing applications. This comprehensive software stack ensures a smooth workflow from development to deployment.

Online fraud detection systems utilize a wide range of software and tools to enhance efficiency and accuracy. Programming languages like R are used for statistical analysis, while Java and Scala are preferred for building scalable systems. Machine learning libraries such as TensorFlow, PyTorch, H2O.ai, and gradient boosting frameworks like XGBoost and LightGBM are commonly employed for advanced modeling. Big data tools like Apache Spark and Hadoop enable the processing of massive datasets, while Kafka supports real-time data streaming for immediate fraud detection. Databases like MongoDB and Cassandra are used for storing unstructured data, while traditional databases like MySQL and PostgreSQL manage structured data. Cloud solutions such as AWS S3, Google Cloud Storage, and Azure provide scalable storage options. Additionally, specialized platforms like SAS Fraud Management are widely adopted for their comprehensive fraud detection and prevention capabilities.

CHAPTER 5

SYSTEM DESIGN

5.1 DATA FLOW DIAGRAM

This diagram represents a step-by-step process of an advanced fraud detection system leveraging machine learning to manage transactions effectively. It begins with Transaction Initiation, where a user initiates a transaction. Following this, Data Collection occurs, gathering all necessary information such as transaction amount, user details, and contextual metadata.

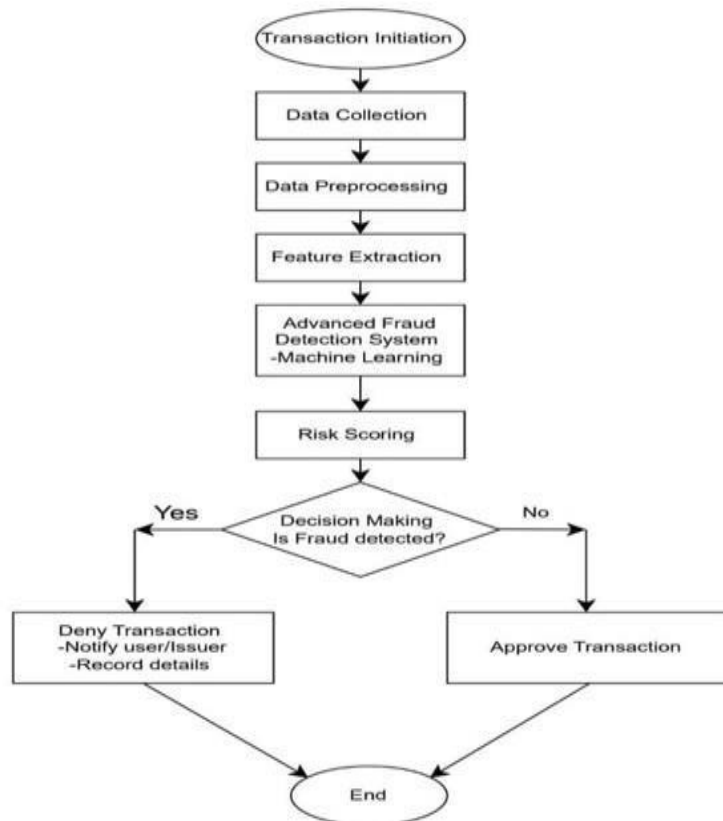


Fig. No. 5.1 Data Flow Diagram

In the Feature Extraction stage, relevant attributes are identified and extracted to highlight potential patterns or irregularities. The processed data is then analyzed by an Advanced Fraud Detection System powered by machine learning, which generates a Risk Scoring based on the likelihood of fraudulent activity

5.2 SYSTEM ARCHITECTURE

This flowchart represents the workflow for a fraud detection system using a machine learning model. The process begins with System Initialization and proceeds to Data Collection, where relevant transaction data is gathered. The collected data undergoes Data Preprocessing to clean and prepare it for modeling. A Random Forest model is used, starting with the Model Training & Testing phase, which splits the dataset for training and validating the model. Afterward, the Model Evaluation step assesses its performance.

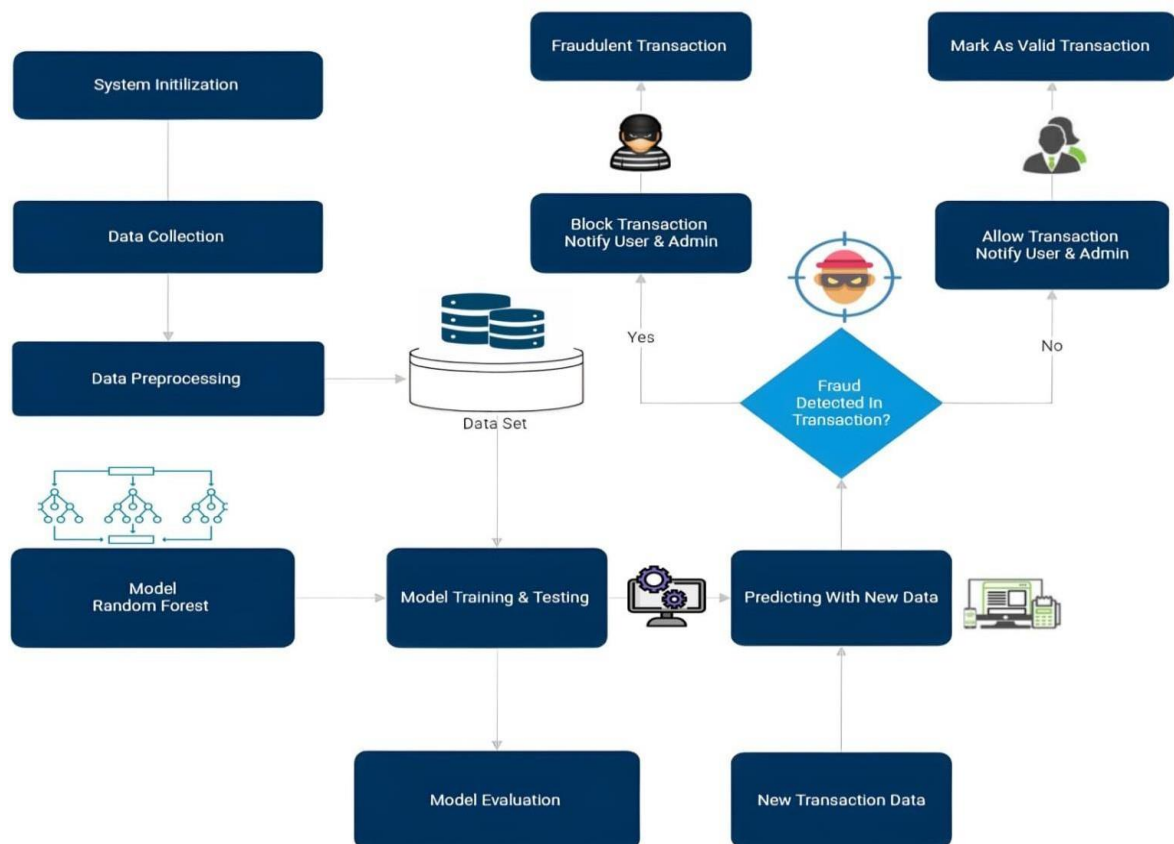


Fig. No. 5.2: System Architecture

For live predictions, New Transaction Data is fed into the system, and the trained model predicts whether a transaction is fraudulent. If 'Fraud Detected in Transaction?' returns "Yes," the system Blocks the Transaction and notifies both the user and admin. If "No," it marks the transaction as valid, allowing it to proceed and notifying the user and admin.

5.3 USE CASE DIAGRAM:

The diagram illustrates a comprehensive workflow for a machine learning-based predictive model system, overseen by an administrator. The process commences with the administrator initiating the workflow, followed by uploading a relevant dataset into the system. This dataset serves as the foundational input for subsequent analysis and prediction tasks. Once the data is uploaded, the system proceeds to extract pertinent features or variables from the dataset, a crucial step that involves data preprocessing and feature selection to ensure the quality and relevance of the information fed into the model.

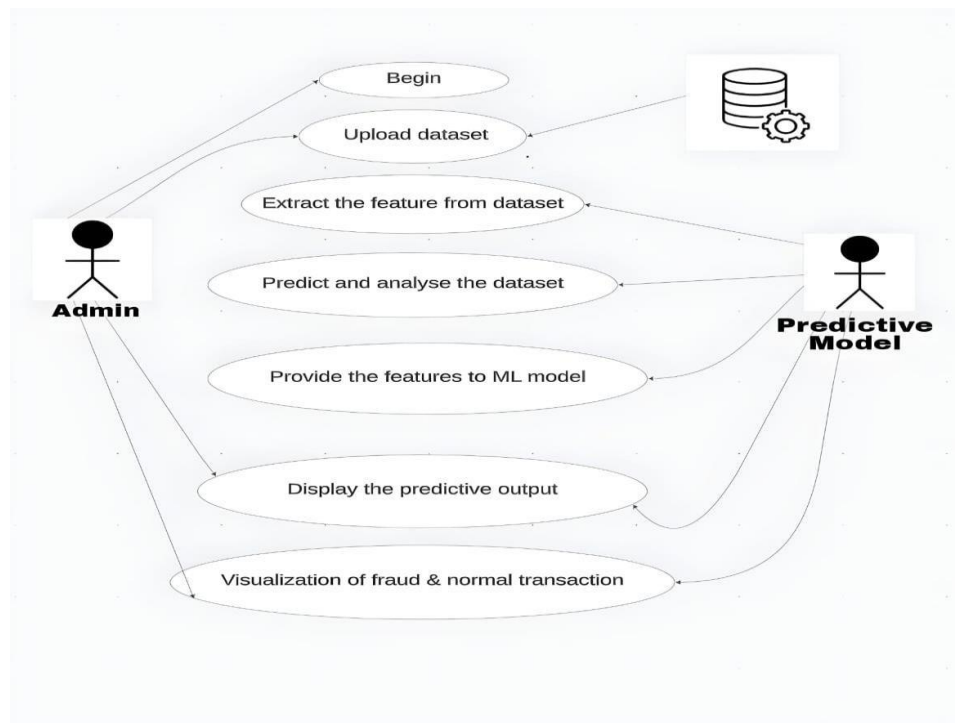


Fig. No. 5.3: Use Case Diagram

These selected features are then supplied to the machine learning model, which constitutes the core component responsible for generating predictions. The model processes the input features to produce predictive outcomes, which the system subsequently analyzes to derive meaningful insights. These predictive results are then displayed to the administrator, highlighting key findings and outcomes.

5.4 CLASS DIAGRAM

This diagram represents a Fraud Detection System with key components and their relationships. At the core, the FraudDetectionEngine analyzes transactions (Transaction) and generates alerts (Alert) if fraudulent activity is detected. The Transaction entity contains attributes such as transaction ID, amount, and timestamp, and is created by users (User).

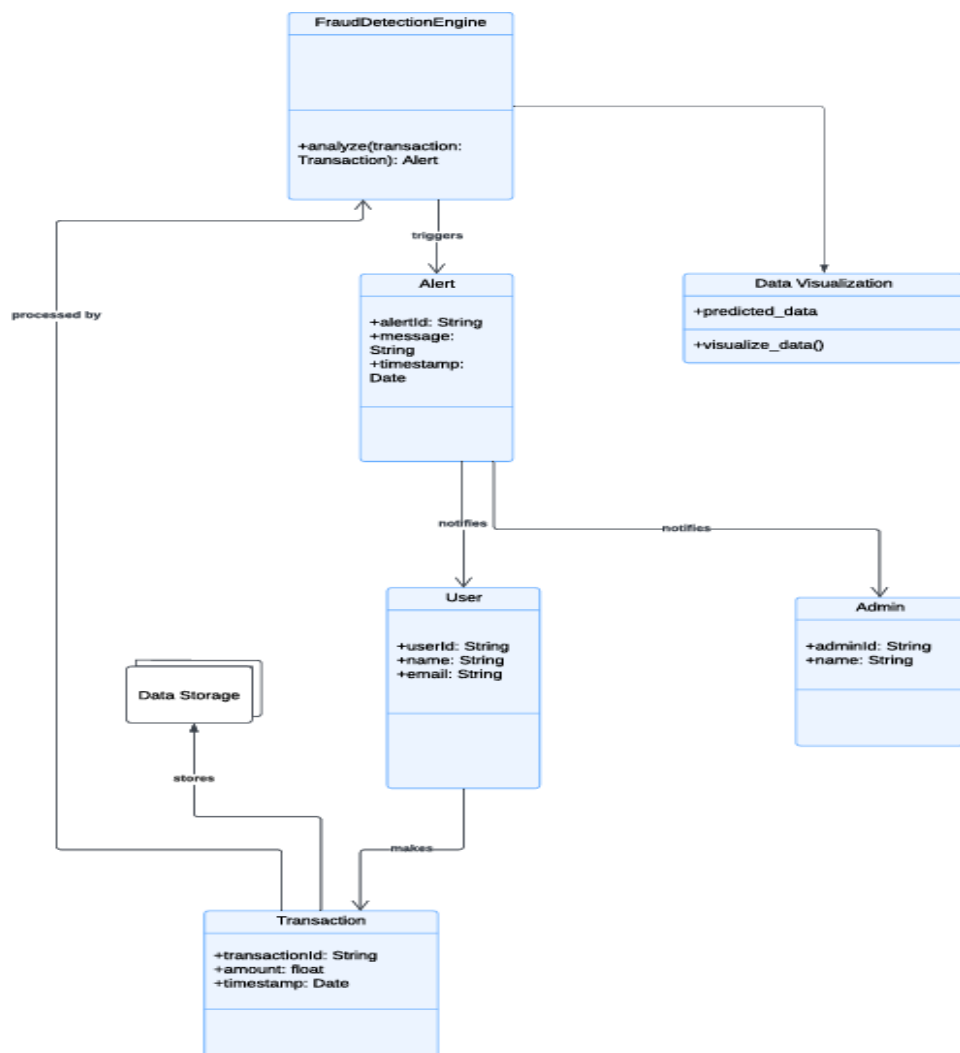


Fig. No. 5.4: Class Diagram

The generated alerts include details like alert ID, a message, and a timestamp. These alerts notify both Users (the individuals involved in transactions) and Admins. Admins can also oversee operations and data through a Data Visualization module, which processes and visualizes predicted data using the `visualize_data()` method.

CHAPTER 6

MODULE DESCRIPTION

6.1 Module 1: Data Collection

Explanation:

Purpose: Gather transaction data that will be analyzed for fraud detection.

Process:

- i) Collect transaction history from personal records, typically stored in PDF format.
- ii) Extract the relevant data from these PDF files, such as transaction date, amount, type, and metadata (e.g., merchant name).
- iii) Convert the extracted data into a structured format (CSV file), which can be easily processed and analyzed.

Outcome: A clean and organized CSV dataset ready for preprocessing

6.2 Module 2: Data Preprocessing

Explanation:

Purpose: Prepare raw data for analysis by cleaning and transforming

Steps Involved:

- i) Handle missing values by filling them with appropriate substitutes or removing incomplete records.
- ii) Encode categorical variables (e.g., transaction type: debit, credit) using techniques like one-hot encoding or label encoding.
- iii) Normalize numerical features like transaction amounts to ensure uniform scaling.
- iv) Identify and remove duplicate records to avoid data redundancy.
- v) Split the data into training and testing sets to evaluate the model later.

Outcome: A preprocessed dataset that ensures reliable and consistent input for the Random Forest algorithm.

6.3 MODULE 3: Synthetic Data Generation

Explanation:

Purpose: Augment the existing dataset by creating synthetic data to improve model robustness and performance, especially when dealing with class imbalance in fraudulent and non-fraudulent transactions.

Process:

- i) **Analyze Data Distribution:** Identify class imbalance in the dataset (e.g., significantly fewer fraudulent transactions)
- ii) **Synthetic Data Creation:** Use techniques like SMOTE (Synthetic Minority Oversampling Technique) to generate synthetic fraudulent samples
 - Ensure the generated data mimics the statistical properties of real-world transactions.
- iii) **Validation of Synthetic Data:** Verify that the synthetic data does not introduce unrealistic patterns or noise into the dataset.
 - Combine the synthetic and original data into a unified dataset.

Outcome: An enhanced dataset that reduces class imbalance and improves the Random Forest model's ability to accurately detect fraudulent transactions

6.4 MODULE 4: Model (Random Forest)

Explanation:

Purpose: Build the detection model using the Random Forest algorithm.

Key Features of Random Forest:

- i) It creates multiple decision trees during training and combines their outputs to make predictions.
- ii) Handles imbalanced data and non-linear relationships effectively.
- iii) Reduces overfitting through averaging (in classification) or voting (in regression).

Outcome: A configured Random Forest model ready for training.

6.5 MODULE 5: Model Training

Explanation:

Purpose: Train a machine learning model, specifically the Random Forest algorithm, to classify transactions as fraudulent or non-fraudulent based on the features in the dataset.

Process:

- i) **Prepare Training Data:** Use the preprocessed and cleaned dataset.
-Split the dataset into training (80%) and validation/testing (20%) sets.
- ii) **Feature Selection:** Identify the most relevant features for fraud detection (e.g., transaction amount, frequency, type, and other metadata).
-Ensure features are scaled or normalized if required by the model.
- iii) **Train the Random Forest Model:** Use the training data to fit a Random Forest model, which creates an ensemble of decision trees to improve prediction accuracy and reduce overfitting.
-Set hyperparameters like the number of trees, maximum depth, and minimum samples per leaf.
- iv) **Hyperparameter Tuning:** Use techniques like grid search or random search to optimize hyperparameters for better model performance.
- v) **Cross-Validation:** Validate the model using k-fold cross-validation to ensure consistent performance across different subsets of the dataset.

Outcome: A trained Random Forest model ready for evaluation, capable of making predictions on new transaction data with high accuracy and reliability.

6.6 MODULE 6: Model Evaluation

Explanation:

Purpose: Assess the performance and reliability of the trained Random Forest model in detecting fraudulent transactions

Process:

- i) Split the Dataset: Use the preprocessed dataset to create training and testing sets, typically using an 80:20 split.
- ii) Evaluate Model on Test Data: Feed the test data into the trained Random Forest model to predict fraud.
- iii) Measure Performance Metrics: Calculate key evaluation metrics such as:
 - Accuracy
 - Precision
 - Recall (Sensitivity)
 - F1 Score

Outcome: A detailed evaluation report highlighting the Random Forest model's effectiveness, strengths, and areas of improvement for fraud detection.

6.7 MODULE 7: Model Prediction

EXPLANATION:

Purpose: Use the trained model to predict whether a new transaction is fraudulent or legitimate.

Steps Involved:

- 1) Input unseen transactional data into the model.
- 2) The Random Forest algorithm analyzes the input features and predicts the likelihood of fraud.
- 3) Classify the transaction as Fraudulent or Legitimate based on the predicted probability threshold (e.g., 0.5).

Outcome: A prediction system that flags suspicious transactions for further review.

6.8 MODULE 8: Visualization

EXPLANATION:

Purpose: Provide insights into the dataset, model performance, and predictions using visual representations.

Key Visuals:

- 1) Feature Importance: Bar chart highlighting the most influential factors in fraud detection.
- 2) Performance Metrics: Graphs (like ROC curve or Precision-Recall curve) to demonstrate model accuracy and reliability.
- 3) Transaction Trends: Line or bar graphs showing the distribution of fraudulent vs. legitimate transactions over time.
- 4) Confusion Matrix Heatmap: A visual summary of the model's performance in categorizing fraud and non-fraud.

Outcome: Clear and interactive visuals to understand the model's results and support decision-making.

CHAPTER 7

CONCLUSION & FUTURE ENHANCEMENT

7.1. CONCLUSION

Online transactional fraud detection has become an essential tool for securing digital financial ecosystems and protecting users from fraudulent activities. By implementing machine learning algorithms such as Random Forest, organizations can effectively identify anomalies, detect suspicious patterns, and reduce the occurrence of fraudulent transactions. Random Forest, with its ensemble-based approach, provides robustness, scalability, and high accuracy, making it a popular choice for such applications. The integration of advanced computational hardware and big data tools further enhances the speed and efficiency of these systems, enabling real-time fraud detection for large-scale operations.

7.2 FUTURE ENHANCEMENT

As we look forward, the future to improve online transactional fraud detection systems, future advancements will focus on integrating more sophisticated machine learning techniques such as Gradient Boosting and deep learning methods like RNNs and LSTMs, which can better identify complex fraud patterns. Real-time detection will be enhanced using streaming tools like Apache Kafka, enabling instantaneous fraud prevention. Behavioral analytics, including user activity and biometric patterns, will provide deeper insights for anomaly detection while minimizing false positives. Explainable AI (XAI) will ensure transparency and regulatory compliance by making detection decisions understandable. Blockchain technology will offer secure, tamper-proof transaction records, adding a layer of trust. Scalability will be achieved through cloud-based platforms like AWS and Google Cloud, ensuring systems can handle growing data and computational demands. Finally, fostering collaboration among financial institutions and sharing threat intelligence will create a unified approach to combating evolving fraud tactics.

APPENDIX 1 SAMPLE CODE

1) Pdf to Csv .py:

```
import pdfplumber
import pandas as pd
import re

def extract_transactions_from_pdf(pdf_path, output_csv_path):
    data = {"Date": [], "Details": [], "Debit": [], "Credit": [], "Balance": [] }
    with pdfplumber.open(pdf_path) as pdf:
        for page_index, page in enumerate(pdf.pages):
            text = page.extract_text()
            if text:
                lines = text.split('\n')
                for line in lines:
                    if is_transaction_line(line):
                        parts = line.split()
                        if len(parts) >= 5:
                            date_match = re.search(r'(\d{1,2})\s([A-Za-z]{3})\s\d{4}', line)
                            if date_match:
                                date = date_match.group(1)
                                data["Date"].append(date)
                            else:
                                data["Date"].append("")
                                data["Details"].append(" ".join(parts[3:-3]))
                                data["Debit"].append(parts[-3])
                                data["Credit"].append(parts[-2])
                                data["Balance"].append(parts[-1])
                        df = pd.DataFrame(data)
```



```

df['Debit'] = pd.to_numeric(df['Debit'], errors='coerce').fillna(0)
df['Credit'] = pd.to_numeric(df['Credit'], errors='coerce').fillna(0)
df['Balance'] = pd.to_numeric(df['Balance'], errors='coerce').fillna(0)
df.to_csv(output_csv_path, index=False)
print(f"Data extracted to {output_csv_path}")
def is_transaction_line(line):
return any(keyword in line for keyword in ["UPI", "TRANSFER", "DEBIT",
"CREDIT"])

```

2) Generate synthetic data.py:

```

import pandas as pd
import numpy as np

def generate_synthetic_features(input_csv, output_csv):
df = pd.read_csv(input_csv)
print("Initial Data:")
print(df.head())
df.columns = df.columns.str.strip()
df['Date'] = df['Date'].astype(str).str.strip()
try:
df['Date'] = pd.to_datetime(df['Date'], format='%d %b %Y', errors='coerce')
except Exception as e:
print(f"Error parsing dates: {e}")
print("NaT values in Date column:")
print(df[df['Date'].isnull()])
df = df.dropna(subset=['Date'])
print("Data after dropping NaT values:")
print(df.head())
df = df.sort_values(by=['Date'])

```

```

df['Transaction_Frequency'] =
df.groupby(df['Date'].dt.date)['Date'].transform('count') - 1

df['Transaction_Time'] = df['Date'] + pd.to_timedelta(np.random.randint(0, 24),
unit='h')

df['Hour_of_Day'] = df['Transaction_Time'].dt.hour

locations = ["Delhi", "Mumbai", "Bangalore", "Chennai", "Trichy", "Kolkata",
"International"]

df['Location'] = np.where(np.random.rand(len(df)) > 0.8, "International",
np.random.choice(locations, len(df)))

df['Transaction_Type'] = df['Details'].apply(lambda x: "UPI" if "UPI" in x else "ATM"
if "ATM" in x else "Other")

df['Fraudulent'] = np.where((df['Debit'] > 1000) & (df['Location'] == "International")
& (df['Hour_of_Day'] > 22), 1, 0)

if df.empty:
print("Warning: The final DataFrame is empty. No data will be saved.")
else:
print("Final DataFrame before saving:")
print(df.head())
df.to_csv(output_csv, index=False)
print(f"Synthetic features added to {output_csv}")

```

3)Preprocess data.py:

```

import pandas as pd

from sklearn.model_selection import train_test_split

from sklearn.preprocessing import StandardScaler, OneHotEncoder

from sklearn.compose import ColumnTransformer

from sklearn.pipeline import Pipeline

```

```

import joblib # To save preprocessor

def load_data(file_path):
    return pd.read_csv(file_path)

def preprocess_data(df, is_predicting=False):
    """
    Preprocess the data for model training or prediction.

    Parameters:
    df: DataFrame containing the data to preprocess
    is_predicting: Whether the function is preprocessing for prediction (defaults to False
    for training)

    Returns:
    X_processed: Processed feature data
    y (optional): Target variable for training
    """

    if not is_predicting:
        X = df.drop(columns=["Fraudulent", "Date", "Transaction_Time"])
        y = df["Fraudulent"]
        return preprocess_features(X), y
    else:
        X = df.drop(columns=["Date", "Transaction_Time"])
        return preprocess_features(X)

def preprocess_features(X):
    """
    Apply preprocessing steps to the features.

    Parameters:
    X: Features to preprocess

    Returns:

```

X_processed: Processed feature data

"""

```
numerical_cols = ["Debit", "Credit", "Balance", "Transaction_Frequency",  
"Hour_of_Day"]
```

```
categorical_cols = ["Details", "Location", "Transaction_Type"]
```

```
numerical_transformer = StandardScaler()
```

```
categorical_transformer = OneHotEncoder(drop='first')
```

```
preprocessor = ColumnTransformer(  

```

```
transformers=[('num', numerical_transformer, numerical_cols),  

```

```
('cat', categorical_transformer, categorical_cols) ])
```

```
pipeline = Pipeline(steps=[('preprocessor', preprocessor)])
```

```
X_processed = pipeline.fit_transform(X)
```

```
joblib.dump(pipeline, 'src/model/preprocessor.joblib')
```

```
return X_processed
```

```
def split_data(X, y, test_size=0.2, random_state=42):
```

```
return train_test_split(X, y, test_size=test_size, random_state=random_state)
```

```
if __name__ == "__main__":
```

```
data_path = r"E:\DESIGN PROJECT 1\fraud_detection_project\data\final_dataset.csv"
```

```
df = load_data(data_path)
```

```
X, y = preprocess_data(df)
```

```
X_train, X_test, y_train, y_test = split_data(X, y)
```

```
print("Data preprocessing complete and split into train and test sets.")
```

4) Training _ evaluate _model

i) train model.py:

```
import joblib

from sklearn.ensemble import RandomForestClassifier

def train_model(X_train, y_train):

    rf_model = RandomForestClassifier(random_state=42)

    rf_model.fit(X_train, y_train)

    joblib.dump(rf_model, 'src/model/random_forest_model.joblib')

    return rf_model

if __name__ == "__main__":

    X_train, X_test, y_train, y_test = joblib.load('src/model/preprocessed_data.joblib')

    rf_model = train_model(X_train, y_train)

    print("Model training complete.")
```

ii) evaluate model.py:

```
from sklearn.ensemble import RandomForestClassifier

from sklearn.model_selection import GridSearchCV

import joblib

def evaluate_model(model, X_test, y_test):

    """
```

Evaluate the model using the test dataset.

Parameters:

- model: Trained classifier model
- X_test: Features for testing

y_test: True labels for testing

Returns:

None

"""

```
from sklearn.metrics import classification_report, confusion_matrix ,y_pred =  
model.predict(X_test)
```

```
print("Classification Report:")
```

```
print(classification_report(y_test, y_pred))
```

```
print("Confusion Matrix:")
```

```
print(confusion_matrix(y_test, y_pred))
```

```
def tune_hyperparameters(X_train, y_train):
```

"""

Tune the hyperparameters of a Random Forest Classifier using GridSearchCV.

Parameters:

- X_train: Features for training
- y_train: Labels for training

Returns:

- best_rf_model: The best Random Forest model found by GridSearchCV

"""

```
print("[INFO] Hyperparameter tuning with GridSearchCV...")
```

```
rf_model = RandomForestClassifier(random_state=42)
```

```
param_grid = {'n_estimators': [100, 200, 300], 'max_depth': [None, 10, 20, 30],
```

```
'min_samples_split': [2, 5, 10], 'min_samples_leaf': [1, 2, 4] }
```

```
grid_search = GridSearchCV(estimator=rf_model, param_grid=param_grid, cv=3,  
n_jobs=-1, verbose=2)
```

```
grid_search.fit(X_train, y_train)
```

```
print(f"Best Hyperparameters: {grid_search.best_params_}")
```

```

best_rf_model = grid_search.best_estimator_
joblib.dump(best_rf_model, 'src/model/best_random_forest_model.joblib')
return best_rf_model

```

5) Main.py:

```

from src.preprocess.preprocess_data import load_data, preprocess_data, split_data
from src.model.train_model import train_model
from src.model.evaluate_model import evaluate_model, tune_hyperparameters
import joblib
import pandas as pd
from src.predict import make_predictions
from src.visualize_predictions import load_predictions, plot_fraudulent_distribution
def main():
    print("[INFO] Loading the final dataset...")
    data_path = "data/final_dataset.csv"
    df = load_data(data_path)
    print("[INFO] Preprocessing the dataset...")
    X, y = preprocess_data(df)
    X_train, X_test, y_train, y_test = split_data(X, y)
    print("[INFO] Preprocessing complete. Saving preprocessed data...")
    joblib.dump((X_train, X_test, y_train, y_test), 'src/model/preprocessed_data.joblib')
    print("[INFO] Training the model...")
    rf_model = train_model(X_train, y_train)
    print("[INFO] Model training complete. Saving the model...")
    joblib.dump(rf_model, 'src/model/random_forest_model.joblib')
    print("[INFO] Evaluating the model...")
    evaluate_model(rf_model, X_test, y_test)

```

```

print("[INFO] Starting hyperparameter tuning...")
best_rf_model = tune_hyperparameters(X_train, y_train)
print("[INFO] Best Hyperparameters:", best_rf_model.get_params())
print("[INFO] Loading the best model after tuning...")
best_rf_model = joblib.load('src/model/best_random_forest_model.joblib')
print("[INFO] Evaluating the best model...")
evaluate_model(best_rf_model, X_test, y_test)
print("[INFO] Making predictions on new transaction data...")
make_predictions()
print("[INFO] Visualizing the results of the predictions...")
predictions_file = 'data/predictions.csv'
predictions_df = load_predictions(predictions_file)
plot_fraudulent_distribution(predictions_df)
if __name__ == "__main__":
    main()

```

6) Predict .py:

```

import pandas as pd
import joblib
preprocessor = joblib.load('src/model/preprocessor.joblib')
best_rf_model = joblib.load('src/model/best_random_forest_model.joblib')
def make_predictions():
    new_data = pd.read_csv('data/new_transaction_data.csv')
    print(f"Original new data shape: {new_data.shape}")
    X_new = new_data.drop(columns=["Date", "Transaction_Time"])
    print(f"New data shape after dropping non-feature columns: {X_new.shape}")
    X_new_processed = preprocessor.transform(X_new)

```



```

print(f"Processed new data shape: {X_new_processed.shape}")
if X_new_processed.shape[1] != best_rf_model.n_features_in_:
    print(f"Feature mismatch! Expected {best_rf_model.n_features_in_} features, but got {X_new_processed.shape[1]}")
    return
y_pred = best_rf_model.predict(X_new_processed)
predictions_df = pd.DataFrame(y_pred, columns=["Fraudulent_Prediction"])
predictions_df.to_csv("data/predictions.csv", index=False)
print("[INFO] Predictions saved to 'data/predictions.csv'.")
if __name__ == "__main__":
    make_predictions()

```

7) Visualize prediction.py:

```

import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns

def load_predictions(file_path):
    print(f"[INFO] Loading predictions from {file_path}...")
    predictions_df = pd.read_csv(file_path)
    print("[INFO] Columns in predictions file:", predictions_df.columns)
    return predictions_df

def plot_fraudulent_distribution(predictions_df):
    print("[INFO] Plotting fraudulent distribution...")
    fraud_count = predictions_df['Fraudulent_Prediction'].value_counts()
    plt.figure(figsize=(8, 5))
    sns.barplot(x=fraud_count.index, y=fraud_count.values, palette='viridis')

```

```

plt.title('Predicted Fraudulent vs Non-Fraudulent Transactions')
plt.xlabel('Fraudulent (1) / Non-Fraudulent (0)')
plt.ylabel('Count')
plt.xticks([0, 1], ['Non-Fraudulent', 'Fraudulent'], rotation=0)
plt.show()

def main():
    predictions_file = 'data/predictions.csv'
    predictions_df = load_predictions(predictions_file)
    if 'Fraudulent_Prediction' in predictions_df.columns:
        plot_fraudulent_distribution(predictions_df)
    else:
        print("[ERROR] 'Fraudulent_Prediction' column not found in the predictions file.")
    if __name__ == "__main__":
        main()

```

8) Index.html

```

<!DOCTYPE html>

<html lang="en">

<head>

<meta charset="UTF-8">

<meta name="viewport" content="width=device-width, initial-scale=1.0">

<title>Fraud Detection Project</title>

<link rel="stylesheet" href="static/css/index.css">

<link
href="https://fonts.googleapis.com/css2?family=Montserrat:wght@400;600&display=
swap" rel="stylesheet">

</head>

```

```
<body>

<!-- Header Section -->

<header>

<h1>Fraud Detection Project</h1>

<p>Your reliable system for identifying fraudulent transactions.</p>

</header>

<!-- Hero Section -->

<section class="hero">

<div class="hero-content">

<h1>Welcome to the Future of Fraud Prevention</h1>

<p>Empowering businesses to combat fraudulent activities with intelligent algorithms
and data-driven insights.</p>

<a href="/dashboard" class="btn">Get Started</a>

</div>

</section>

<!-- Footer Section -->

<footer>

<p>&copy; 2024 Fraud Detection Project. All rights reserved.</p>

<p>

<a href="/privacy-policy">Privacy Policy</a> |

<a href="/terms">Terms of Service</a>

</p>

</footer>

</body>

</html>
```

9) Dashboard.html

```
<!DOCTYPE html>

<html lang="en">

<head>

<meta charset="UTF-8">

<meta name="viewport" content="width=device-width, initial-scale=1.0">

<title>Dashboard</title>

<link rel="stylesheet" href="{{ url_for('static', filename='css/dashboard.css') }}">

<link rel="stylesheet" href="https://cdnjs.cloudflare.com/ajax/libs/font-
awesome/6.0.0/css/all.min.css" integrity="sha512-..." crossorigin="anonymous"
referrerpolicy="no-referrer" />

</head>

<body>

<div class="container">

<header class="hero">

<h1>Welcome to Fraud Detection Dashboard</h1>

<p>Navigate through the sections or upload your transaction PDFs for analysis.</p>

<button class="btn" onclick="scrollToSection('upload')">Get Started</button>

</header>

<nav class="navigation">

<ul>

<li><a href="#upload" onclick="scrollToSection('upload')">Upload</a></li>

<li><a href="#about" onclick="scrollToSection('about')">About</a></li>

<li><a href="#contact" onclick="scrollToSection('contact')">Contact</a></li>

</ul> </nav><section id="upload" class="section">
```

<h2>Upload Your Transaction PDF</h2>

<p>Ensure your PDF meets the following criteria:</p>

The PDF file must be unlocked.

Transaction date must be in the format: YYYY-MM-DD.

Ensure data consistency for accurate predictions.

<form action="/upload" method="POST" enctype="multipart/form-data">

<label for="file" class="upload-label">

<i class="fa fa-upload"></i> Choose PDF File

</label>

<input type="file" id="file" name="file" accept=".pdf">

<button type="submit" class="btn">Upload</button>

</form>

</section>

<section id="about" class="section">

<h2>About the Project</h2>

<p>

This project aims to detect fraudulent transactions efficiently using machine learning.

Our system leverages advanced algorithms to identify unusual patterns, ensuring financial security.

</p>

</section>

<section id="contact" class="section">

<h2>Contact Us</h2>

<form action="/contact" method="POST">

<div class="form-group">

```

<label for="name">Name</label>

<input type="text" id="name" name="name" required>

</div>

<div class="form-group">

<label for="email">Email</label>

<input type="email" id="email" name="email" required>

</div>

<div class="form-group">

<label for="message">Message</label>

<textarea id="message" name="message" rows="4" required></textarea>

</div>

<button type="submit" class="btn">Send</button>

</form>

</section>

</div>

<script>

function scrollToSection(sectionId) {

document.getElementById(sectionId).scrollIntoView({ behavior: 'smooth' });

}

function showSuccessPopup() {

alert("File has been successfully uploaded!");

window.location.href = "/analysis"; // Redirect to analysis page

}

</script>

</body>

</html>

```

10) Analysis.html

```
<!DOCTYPE html>

<html lang="en">

<head>

<meta charset="UTF-8">

<meta name="viewport" content="width=device-width, initial-scale=1.0">

<title>Analysis - Fraud Detection</title>

<link rel="stylesheet" href="{{ url_for('static', filename='css/analysis.css') }}">

<link rel="stylesheet" href="https://cdnjs.cloudflare.com/ajax/libs/font-
awesome/6.0.0/css/all.min.css" integrity="sha512-..." crossorigin="anonymous"
referrerpolicy="no-referrer" />

</head>

<body>

<div class="container">

<!-- Header Section -->

<header class="hero">

<h1>Transaction Analysis</h1>

<p>Analyze your uploaded transaction data for potential fraudulent activity.</p>

</header>

<!-- Content Section -->

<section class="analysis-results">

<h2>Uploaded File: {{ filename }}</h2>

<p>The file has been successfully uploaded. You can now visualize the results of the
analysis and explore the findings.</p>

<!-- Visualize Button -->

<button class="btn visualize-btn" onclick="visualizeResults()">Visualize the
```

Results</button>

</section>

<!-- Footer Section -->

<footer>

<p>© 2024 Fraud Detection Project. All Rights Reserved.</p>

</footer>

</div>

<script>

function visualizeResults() {

alert("Visualizing the results... (You can replace this with actual functionality)");

}

</script>

</body>

</html>

APPENDIX 2 SCREENSHOTS

final_dataset.csv X	
C: > Users > vigna > AppData > Local > Temp > 888c342e-04f7-40eb-b306-496b70788e90_fraud_detection_project[1].zip.e90 > fraud_detection_project > data > final_dataset.csv	
1	Date,Details,Debit,Credit,Balance,Transaction_Frequency,Transaction_Time,Hour_of_Day,Location,Transaction_Type,Fraudulent
2	2023-06-11,BY TRANSFER -,0.0,0.0,1000.0,0,2023-06-11 08:00:00,8,Mumbai,Other,0
3	2023-07-03,WITHDRAWAL TRANSFER,0.0,0.0,2650.0,4,2023-07-03 08:00:00,8,International,Other,0
4	2023-07-03,BY TRANSFER -,0.0,0.0,2650.0,4,2023-07-03 08:00:00,8,Chennai,Other,0
5	2023-07-03,BY TRANSFER -,0.0,0.0,2500.0,4,2023-07-03 08:00:00,8,Mumbai,Other,0
6	2023-07-03,WITHDRAWAL TRANSFER INR 50.00 -,0.0,0.0,2450.0,4,2023-07-03 08:00:00,8,Mumbai,Other,0
7	2023-07-03,WITHDRAWAL TRANSFER,0.0,0.0,2400.0,4,2023-07-03 08:00:00,8,International,Other,0
8	2023-07-05,BY TRANSFER -,0.0,0.0,3400.0,3,2023-07-05 08:00:00,8,Kolkata,Other,0
9	2023-07-05,WITHDRAWAL TRANSFER,0.0,0.0,3400.0,3,2023-07-05 08:00:00,8,International,Other,0
10	2023-07-05,BY TRANSFER UPI RVSL -,0.0,0.0,3400.0,3,2023-07-05 08:00:00,8,Kolkata,UPI,0
11	2023-07-05,WITHDRAWAL TRANSFER,0.0,0.0,3400.0,3,2023-07-05 08:00:00,8,International,Other,0
12	2023-07-25,CIUB0000201/Mr KANNAN,0.0,0.0,20000.0,2,2023-07-25 08:00:00,8,International,Other,0
13	2023-07-25,UNCOLL CHRG DT: INR 30.03 -,0.0,0.0,22777.94,2,2023-07-25 08:00:00,8,Delhi,Other,0
14	2023-07-25,BY TRANSFER NPCI -,0.0,0.0,22800.0,2,2023-07-25 08:00:00,8,Chennai,Other,0
15	2023-07-26,utib0000553/Google India INR 30.00 -,0.0,0.0,2717.94,9,2023-07-26 08:00:00,8,Trichy,Other,0
16	2023-07-26,IDIB000G011/Mr INR 60.00 -,0.0,0.0,2657.94,9,2023-07-26 08:00:00,8,International,Other,0
17	2023-07-26,IDIB000G011/Mr Kamalesh INR 6.00 -,0.0,0.0,2621.94,9,2023-07-26 08:00:00,8,Bangalore,Other,0
18	2023-07-26,IDIB000G011/Mr Kamalesh INR 10.00 -,0.0,0.0,2611.94,9,2023-07-26 08:00:00,8,Delhi,Other,0
19	2023-07-26,UTIB0000000/Store/XXXXX INR 20.00 -,0.0,0.0,2591.94,9,2023-07-26 08:00:00,8,Delhi,Other,0
20	2023-07-26,IOBA0000520/DINESH INR 40.00 -,0.0,0.0,2551.94,9,2023-07-26 08:00:00,8,International,Other,0
21	2023-07-26,TRAN DATE -(MMD) 0726 INR 100.00 -,0.0,0.0,2451.94,9,2023-07-26 08:00:00,8,Kolkata,Other,0
22	2023-07-26,ICIC0DC0099/ZOMATO INR 100.70 -,0.0,0.0,2351.24,9,2023-07-26 08:00:00,8,Kolkata,Other,0
23	2023-07-26,PYTM0123456/ATHAVAN INR 30.00 -,0.0,0.0,2747.94,9,2023-07-26 08:00:00,8,Mumbai,Other,0
24	2023-07-26,IDIB000G011/Mr Kamalesh INR 30.00 -,0.0,0.0,2627.94,9,2023-07-26 08:00:00,8,International,Other,0
25	2023-07-27,CNRB0000033/S INR 1.00 -,0.0,0.0,2350.24,1,2023-07-27 08:00:00,8,Trichy,Other,0
26	2023-07-27,UTIB0000000/Zomato INR 132.20 -,0.0,0.0,2218.04,1,2023-07-27 08:00:00,8,Mumbai,Other,0
27	2023-07-28,UPI SETTLEMENT A/C - INR 40.00,0.0,0.0,2258.04,0,2023-07-28 08:00:00,8,International,UPI,0
28	2023-07-29,IDIB000G011/Mr Kamalesh INR 100.00 -,0.0,0.0,2158.04,1,2023-07-29 08:00:00,8,Delhi,Other,0
29	2023-07-29,PYTM0123456/MEGALA INR 20.00 -,0.0,0.0,2138.04,1,2023-07-29 08:00:00,8,Delhi,Other,0
30	2023-07-30,utib0000553/Google India INR 79.00 -,0.0,0.0,2059.04,0,2023-07-30 08:00:00,8,Chennai,Other,0
31	2023-07-31,ICIC0DC0099/EURONETG INR 30.00 -,0.0,0.0,2199.04,3,2023-07-31 08:00:00,8,Mumbai,Other,0
32	2023-07-31,CIUB0000201/Mr KANNAN - INR 200.00,0.0,0.0,2259.04,3,2023-07-31 08:00:00,8,Mumbai,Other,0
33	2023-07-31,PYTM0123456/ATHAVAN INR 30.00 -,0.0,0.0,2229.04,3,2023-07-31 08:00:00,8,Chennai,Other,0
34	2023-07-31,MIN BAL CHGS INR 360.00 -,0.0,0.0,1839.04,3,2023-07-31 08:00:00,8,Bangalore,Other,0
35	2023-08-01,utib0000553/Google India INR 60.00 -,0.0,0.0,1779.04,1,2023-08-01 08:00:00,8,Bangalore,Other,0

Fig.No.A.2.1 Datasets

```

(venv) E:\DESIGN PROJECT 1\fraud_detection_project>python main.py
[INFO] Loading the final dataset...
[INFO] Preprocessing the dataset...
[INFO] Preprocessing complete. Saving preprocessed data...
[INFO] Training the model...
[INFO] Model training complete. Saving the model...
[INFO] Evaluating the model...
Classification Report:

```

	precision	recall	f1-score	support
0	1.00	1.00	1.00	65
1	1.00	1.00	1.00	20
accuracy			1.00	85
macro avg	1.00	1.00	1.00	85
weighted avg	1.00	1.00	1.00	85

```

Confusion Matrix:
[[65  0]
 [ 0 20]]
[INFO] Starting hyperparameter tuning...
[INFO] Hyperparameter tuning with GridSearchCV...
Fitting 3 folds for each of 108 candidates, totalling 324 fits
[CV] END max_depth=None, min_samples_leaf=1, min_samples_split=2, n_estimators=100; total time= 0.4s
[CV] END max_depth=None, min_samples_leaf=1, min_samples_split=2, n_estimators=100; total time= 0.4s
[CV] END max_depth=None, min_samples_leaf=1, min_samples_split=2, n_estimators=100; total time= 0.4s
[CV] END max_depth=None, min_samples_leaf=1, min_samples_split=2, n_estimators=200; total time= 0.8s
[CV] END max_depth=None, min_samples_leaf=1, min_samples_split=2, n_estimators=200; total time= 0.8s
[CV] END max_depth=None, min_samples_leaf=1, min_samples_split=2, n_estimators=200; total time= 0.7s
[CV] END max_depth=None, min_samples_leaf=1, min_samples_split=2, n_estimators=300; total time= 1.1s
[CV] END max_depth=None, min_samples_leaf=1, min_samples_split=5, n_estimators=100; total time= 0.3s
[CV] END max_depth=None, min_samples_leaf=1, min_samples_split=5, n_estimators=100; total time= 0.3s
[CV] END max_depth=None, min_samples_leaf=1, min_samples_split=5, n_estimators=100; total time= 0.3s
[CV] END max_depth=None, min_samples_leaf=1, min_samples_split=2, n_estimators=300; total time= 1.2s
[CV] END max_depth=None, min_samples_leaf=1, min_samples_split=2, n_estimators=300; total time= 1.2s
[CV] END max_depth=None, min_samples_leaf=1, min_samples_split=5, n_estimators=200; total time= 0.7s

```

Fig.No.A.2.2 Model evaluation and hyper parameter tuning

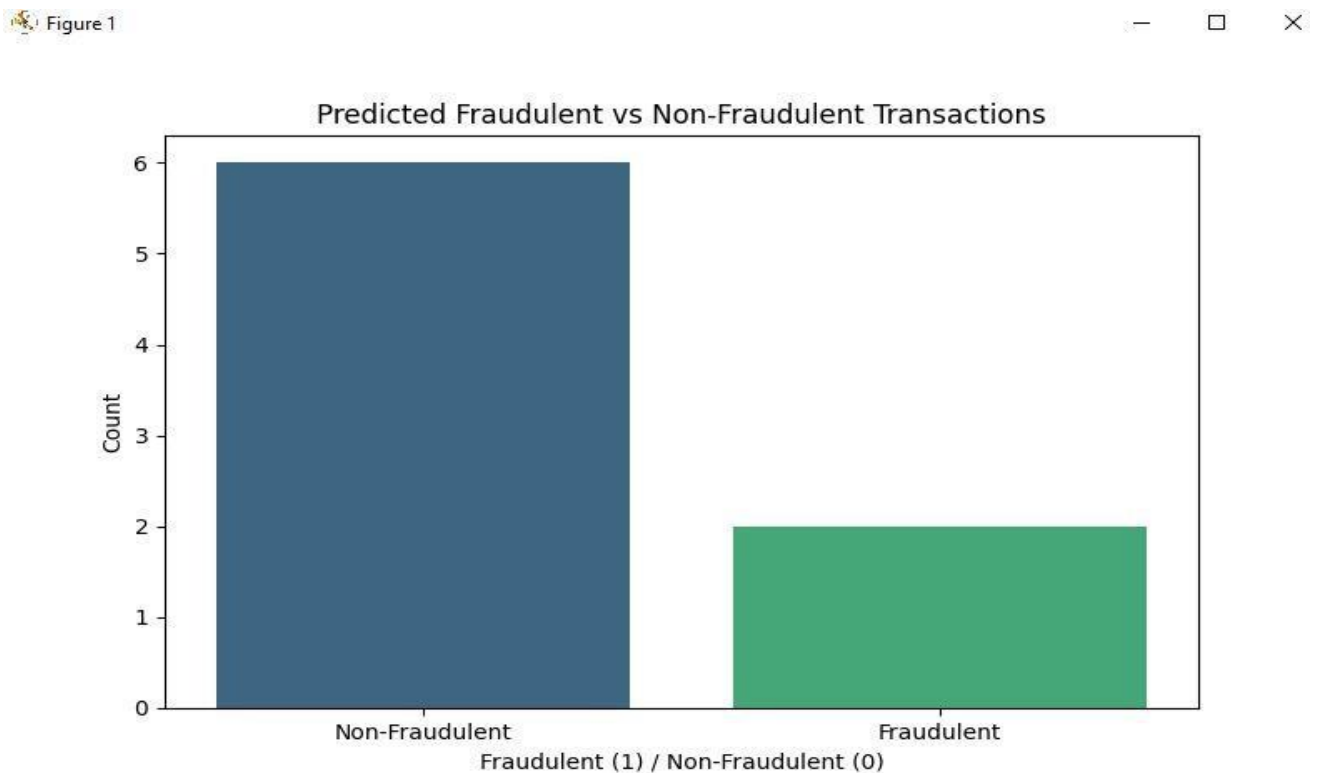


Fig.No.A.2.3 Visualization

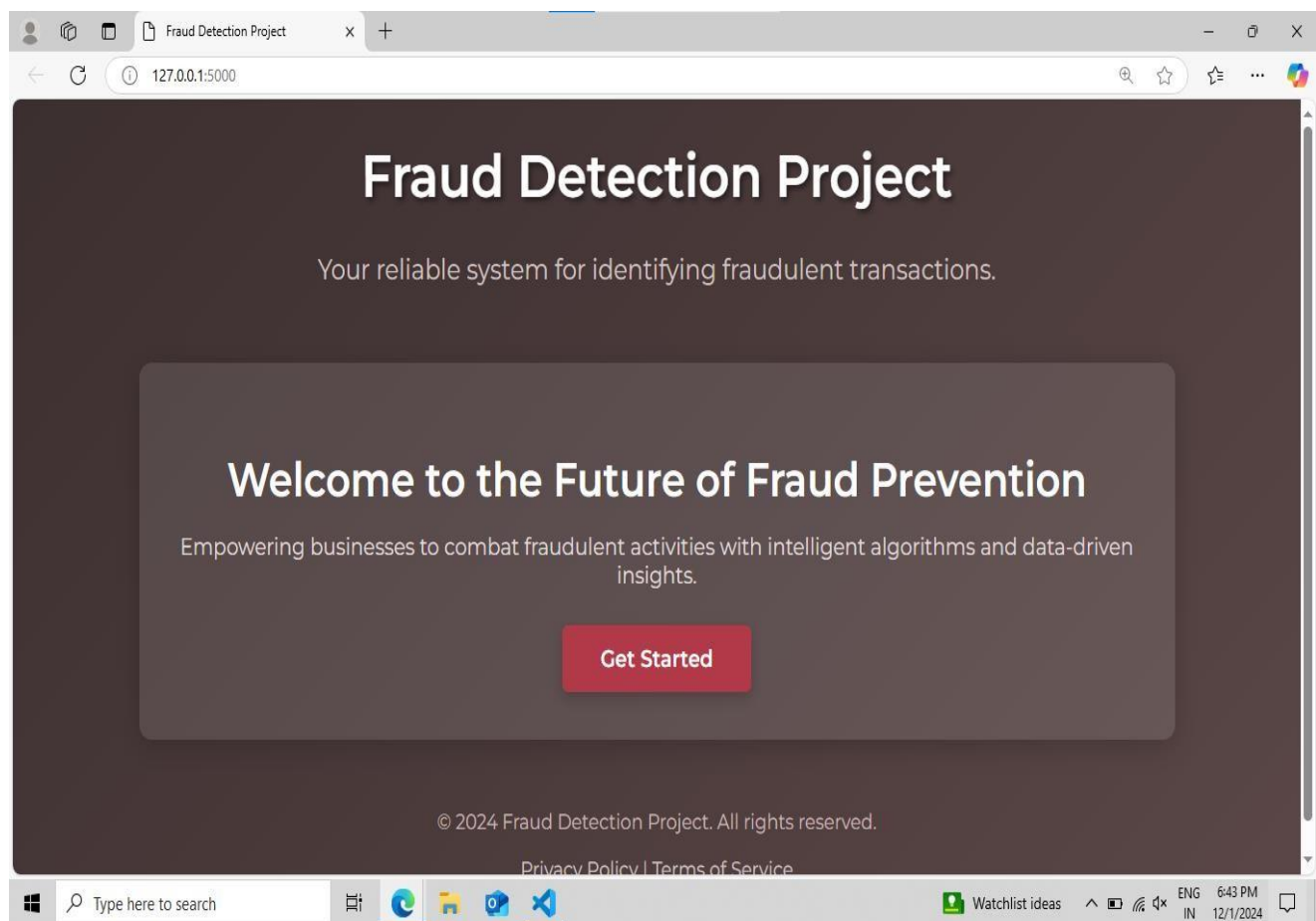


Fig.No.A.2.4 Home page

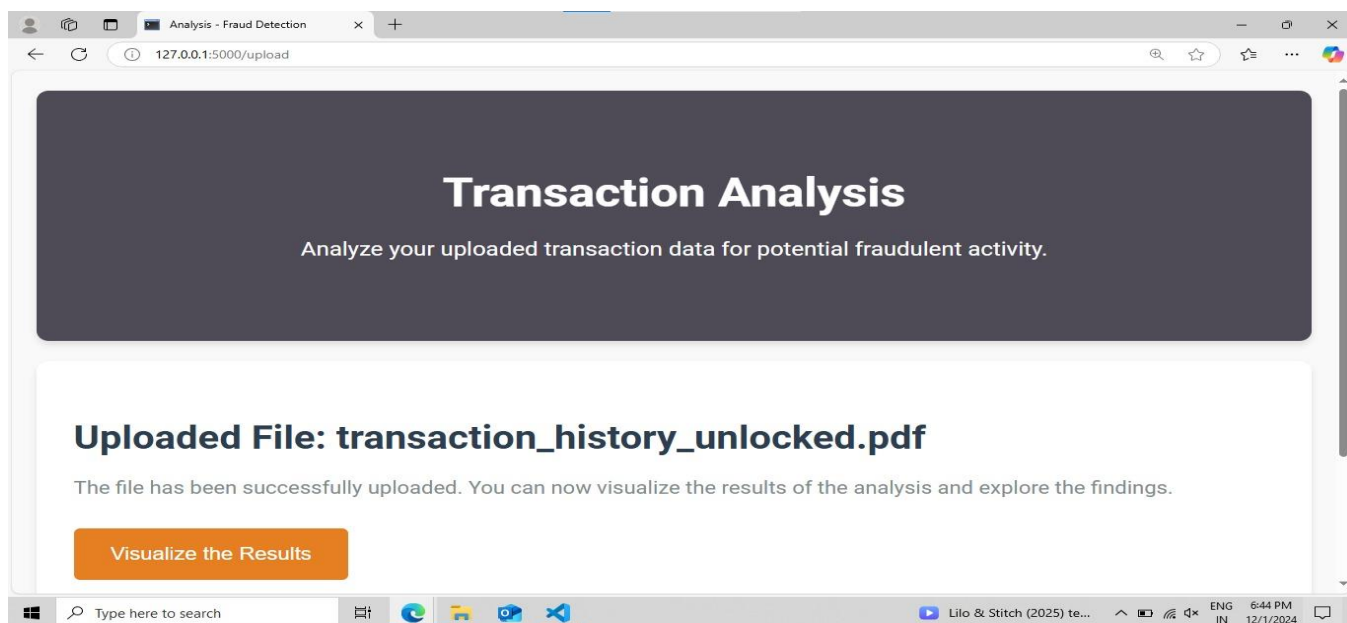


Fig.No.A.2.5 Pdf uploaded acknowledgement page

REFERENCES

1. Hejazi, M., & Singh, Y. P. (1995). *Credit Data Fraud Detection using Kernel Methods with Support Vector Machine*. Journal of Advanced Computer Science and Technology Research, 2, 35–49.
2. M. Syeda, Y.Q. Zhang, Y. Pan, Parallel granular neural networks for fast credit card fraud detection, in: Proceedings of the IEEE International Conference on Fuzzy Systems, 1997, pp. 572–577.
3. Curet, O. and Jackson, M. (2000) Tackling cognitive biases in the detection of top management fraud with the use of case-based reasoning. Proceedings of Expert Systems '95 ed. Macintosh, A. and Cooper, C. pp.223-236, 1995.
4. Aleskerov, E., Freisleben B., and Rao B.(2002). CARDWATCH: A Neural Network Based Database Mining System for Credit Card Fraud Detection. Computational Intelligence for Financial Engineering, Proceedings of the IEEE/IAFE, 220-226.
5. Vladimir Vapnik, "The Nature of Statistical Learning Theory", 2012, Springer Verlag N.Y., 189 p.